SEASUG Paper 102-2026

Reviving Institutional Knowledge: LLM-Generated Documentation for Legacy SAS® Production Code

Patrick Long and Brandon Harris, Office of Employment and Unemployment Statistics, US Bureau of Labor Statistics

ABSTRACT

Many mature SAS shops run production code that nobody currently on staff wrote. The macros work, the output validates, but occasionally the reasoning behind programming steps and dataset variable names are not easily inferred. This paper reports a controlled comparison of three ways to regenerate that lost documentation automatically: (1) a small open-weight language model running zero-shot on a modest on-premises server; (2) that same model after a QLoRA fine-tune on paired examples of legacy programs and hand-written documentation; and (3) a frontier-scale model given structured, read-only access to SAS metadata through a small set of purpose-built tools, or "skills." In every configuration SAS can remain the system of record: the model writes JSON, and the JSON can land back into SAS data sets. Results are reported on variable coverage, macro parameter recall, data set lineage, review effort, and the metric that decides whether any of this is usable in a production environment — the rate of confident, unsupported assertions.

INTRODUCTION

Legacy SAS code is durable in a way that its documentation is not. A production stream that has run monthly for twenty years accumulates macro variables whose purpose is inferable only from their downstream effect and datasets and column names whose lineage exist as folklore. When the last person who remembers the design retires, the code becomes something the organization operates but no longer understands. Large language models are an obvious candidate for reconstructing that documentation, but also a risky one.

When tasked with describing a previously unseen program, a large language model can generate documentation that is fluent, well-formatted, and seemingly plausible. However, the model may lack sufficient justification for its specific claims, resulting in documentation that may not be grounded in the actual code or metadata. This paper will evaluate three approaches to the problem using LLMs in practical ways to automatically generate structured documentation from legacy SAS programs.

WHAT COUNTS AS DOCUMENTATION

We defined three artifacts in advance, each with a fixed JSON schema:

1. Variable dictionary — for every variable in every output data set: name, type, length, format, label, derivation, valid values, and source reference.
2. Macro parameter reference — for every macro: positional and keyword parameters, defaults, and calling programs.
3. Program Summary — description, inputs and outputs of a program.

Every field in every schema carries a required source element. A field with no defensible source must be emitted as UNKNOWN. This single rule does most of the work in the sections that follow.

ARCHITECTURE: SAS STAYS INVOLVED IN THE PROCESS

Two implementation paths were prototyped. PROC PYTHON keeps SAS in control: the SAS program calls out to Python, which calls the model, and the response returns to the DATA step. SASPy inverts the relationship, with a Python orchestrator driving a SAS session configured to your SAS engine. SASpy was selected for the production pattern because it does not require SAS 9.4 M9 or higher like proc python does and will thus be more reproducible for other SAS users. SASpy can write to SAS datasets, retaining

these stores as the system of record and they can alternatively be written out to json which can easily be written to SAS datasets using PROC JSON. Further, SAS programming language snippets are added to the skills section of one of the configurations below

THE THREE CONFIGURATIONS

With each configuration, we've shared the source code in a GitHub® repository linked at the bottom of this paper. The setup uses Google® Colab and SAS® OnDemand for Academics (ODA), with the aim that these configurations are fully reproducible. For the frontier model, a subscription will be needed to make use of them fully. Specifically, we used Claude® Opus 5 as our frontier model. Conceptually configuration 1 and 2 are demonstrated through Google® Colab but are designed to run air-gapped for users who don't want to be reliant on a frontier model.

CONFIGURATION 1 — BASELINE: SMALL OPEN-WEIGHT MODEL, ZERO-SHOT

A quantized 4B-parameter open-weight model (Gemma-class), Gemma 4-E2B-it, runs without a GPU. It receives the program text and the target schema and is asked to fill it in. No tools, no examples, no retrieval. This is a configuration most analysts can improvise on their own and with minimal computing resources.

In this intentionally limited model, we added a guardrail, implementing a script named guardrail.py, that performs a post static scan of the SAS code to flag elements the model may have hallucinated, helping account for known issues we're likely to encounter and better steer the model. It's plausible for developers to extend this further but with access to a GPU server there's better performance to be gained with less work on the developers' part.

CONFIGURATION 2 — QLORA FINE-TUNE OF THE BASELINE MODEL

QLoRA (Dettmers et al. 2023) freezes the base model in 4-bit precision and trains small low-rank adapter matrices alongside it, making a fine-tune feasible on one GPU. The training corpus is twenty deliberately poorly written production programs paired with hand-written data dictionaries, that is, examples of the input and of exactly what it should become chunked into roughly 580 training pairs.

Hyperparameter	Value	Notes
Base model	[Gemma 4-E2B-it, 4-bit]	NF4 quantization
Adapter rank / alpha	[16 -> 32] / [32 -> 64]	attention + MLP projections
Learning rate	[2e-4]	cosine decay
Epochs / batch	[3 -> 2] / [4 -> 1]	grad. accumulation [8]
Wall-clock train time	[~1.5 hours]	A100 GPU (slower or faster depending on what you use)

Table 1. Hyperparameter QLoRA fine tune of base model (with gpu access)

Small snippet from qlora_finetune.py using python peft library:

```
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training


model = AutoModelForCausalLM.from_pretrained(
          args.model, revision=args.revision, quantization_config=bnb
   )
model.config.use_cache = False
model = prepare_model_for_kbit_training(model)


lora = LoraConfig(
    r=32, lora_alpha=64, lora_dropout=0.05, task_type="CAUSAL_LM",
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
)
model = get_peft_model(model, lora)
```

It is important to note what this kind of fine-tune does: the adapters bias the model toward the shape of the answer, that is, style, field ordering, terminology, by nudging weights toward the training outcome, iteration by iteration (Hu et al. 2021). They do not add knowledge of your code, and they do not raise the base model's reasoning ceiling (Dettmers et al. 2023). In essence, 4B model with adapters is still a 4B model. The expected gain is in format reliability and tone, not in inference about unfamiliar code or macro logic.

CONFIGURATION 3 — FRONTIER MODEL PLUS SKILLS

The third configuration uses a frontier-scale model and makes use of the ability to provide it with skill specific to this task. The model works agentically by querying the dictionary tables via PROC SQL, parsing the source, and only then writes out.

The frontier configuration is given these two read-only tools in its skills. Whatever it writes must be traced back to one of them:

- DICTIONARY tables via PROC SQL for cross-library structure — which data sets exist, which share key variables, and where a name is reused with different attributes.
- A custom Python parser that indexes SAS source and returns macro definitions, parameter lists, %LET assignments, and the line numbers where each variable is assigned.

Note: PROC CONTENTS might seem a natural fit for adding to skills, but it is a human-readable tool for variable-level metadata. Whereas dictionary.columns/tables are tables and can be pulled straight into a dataframe object using the saspy library's .sd2df() method.

Our source code and skills for this config were developed to use Claude Opus 5 for the frontier model. Other frontier models typically have a different setup for their skills file(s). Below is an example snippet added to .claude/skills/ making use of the DICTIONARY tables via PROC SQL for key variables and metadata in the work library:

```
proc sql noprint;
  create table work._sdgcols as
  select memname, name, type, length, format, informat, label, varnum
  from dictionary.columns
```

```
  where libname='WORK'
  order by memname, varnum;

  create table work._sdgtabs as
  select memname, nobs, nvar, memlabel as table_label
  from dictionary.tables
  where libname='WORK' and memtype='DATA';
quit;
```

EVALUATION METHODOLOGY

To move beyond single anecdotal worked examples, three documentation-generation configurations were scored against a common held-out evaluation set of 20 synthetic-but-realistic legacy SAS programs. These were built with Mistral Large 3, since we're using a Claude model for the frontier model we wanted to use another LLM provider for the synthetic testing data. Because each program and its paired "gold" documentation file are generated from the same underlying specification, the the gold labels are correct by construction rather than being hand-authored afterward. This approach enables fully automatic scoring and provides exact, unambigous references for variables, macros and datasets without exposing production code or relying on subjective annotation, though it may miss some of the complexity, conventions and interdependencies found in real productions environments. Further, the prompting for development of these programs called for them to be deliberately written the way real legacy SAS accumulates flaws with no, or minimal headers or inline comments, abbreviated variable names (employment_level → EMPL or EM, etc.), cryptic macro parameters (p, lb, thr, dbg), suppressed log detail via options nomprint nosymbolgen, hardcoded paths and reference periods, and throwaway intermediate datasets.

The three configurations were compared, each prompted for the identical output schema: variable dictionary, macro-parameter reference, and program summary. The comparison attempts, therefore, to isolate the effect of model and grounding rather than differences in what was asked for. All three configs can be run using Google® Colab with SAS® ODA credentials to write-out SAS datasets.

RESULTS

Each prediction was scored structurally against the gold files, including checks for schema validity, set-comparison F1 for variables, macro parameters, and program inputs/outputs, and a hallucination rate defined as any variable, dataset, or macro-parameter name not found in the source (or, for Config C, in the SASPy-retrieved metadata). These metrics focus on structural fidelity, so they do not capture other semantic errors or issues involving descriptive language in documentation.

Metric	**Config 1 Gemma 4-E2B-it, CPU, zero-shot**	**Config 2 Gemma 4-E2B-it, QLoRA fine-tuned**	**Config 3 Claude Opus 5 + skills**
Schema validity	45%	100%	100%
Variable F1	0.01	0.92	1.00
Macro-parameter F1	0.18	1.00	1.00
Summary and Input/output F1	0.10	1.00	1.00
Hallucination rate	57% (high variability 37-81)	0%	0%
Time per program	88 sec	210 sec	61 sec

Table 2. Illustrative results across the three documentation-generation configurations (placeholder values; n = 20 programs, 3 runs each).

The gap between Config 1 and Config 3 is stark rather than incremental. Config 1 produced schema-valid output for only 9 of the 20 programs, and even restricted to those 9, variable extraction was near-zero in 7 of those 9, but macro-parameter and I/O dataset F1 were inconsistent rather than uniformly zero, ranging from 0 to 1.00 depending on the program. Its 57% average hallucination rate is highly variable, that is, a user could be fortunate and get a low hallucination rate or score highly hallucinated. Given that high hallucination rate, zero-shot prompting of a small model on this task is close to unusable on its own, which is the expected gap Config 2's fine-tuning is intended to test.

Config 2's shows materially higher F1s, a hallucination rate of 0%, well below config 1,without fully reaching Config 3's grounded, near-perfect scores all around, since fine-tuning improves the model's fluency with SAS idioms but does not give it access to ground truth the way SASPy-retrieved metadata does. With further training, or using a higher parameter Gemma-class model, the gap from here to Config 3 could be narrowed. Anecdotally and based on the F1 score Config 2 is a practical locally deployable model for achieving the goal of SAS documentation.

Config 3 is close to the opposite result of Config 1: 100% schema validity and 1.00 F1 on variables, macro parameters, and inputs/outputs across all 20 programs, with zero hallucinated names. The one metric where Config 3 is not a clean 1.00 is a finer-grained check not shown in the summary table — whether a macro parameter's positional-versus-keyword usage was correctly identified — which averaged 0.64, a reminder that the headline numbers, while strong, are not a claim of literal perfection on every dimension scored.

LIMITATIONS

The results should be interpreted with the following caveats in mind:

- Benchmark and training data are intentionally synthetic, enabling exact automated scoring but limits how well the findings may generalize to real-world systems.
- Config 3 is not a direct comparison to 1 and 2; rather it represents a practical approach that might be used when documenting legacy SAS code relative to other practical documentation strategies.
- The hallucination metric captures only a subset of potential errors and does not reflect broader semantic misunderstandings or inaccuracies in how the model might interpret or describe the source code.

CONCLUSION

Large language models show strong potential for assisting with the documentation of legacy SAS programs. QLoRA fine-tuned models also offer a practical alternative for organizations that prefer not to rely on frontier systems or would prefer air-gapped systems. Continued evaluation of emerging frontier models and exploration of additional capabilities may further increase the value of generated documentation, if these are the preferred tool of an organization. While human review should remain an essential part of the process, using an LLM to produce documentation and data dictionaries can significantly reduce the amount of documentation that must be produced manually for SAS programmers, regardless of how well your programs were originally documented or how descriptive their variable names may be.

REFERENCES

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. 2023. "QLoRA: Efficient Finetuning of Quantized LLMs." arXiv:2305.14314.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. 2021. "LoRA: Low-Rank Adaptation of Large Language Models." arXiv:2106.09685.

Ji, Z., Lee, N., Frieske, R., et al. 2023. "Survey of Hallucination in Natural Language Generation." ACM Computing Surveys 55(12).

SAS Institute Inc. "Accessing SAS Information Using DICTIONARY Tables." SAS® SQL Procedure User's Guide. Cary, NC: SAS Institute Inc.

SAS Institute Inc. “SASPy Documentation.” Accessed September 17, 2026, at gitthub.com/sassoftware/saspy.

RECOMMENDED READING

- *Full Source Code used in this paper - https://github.com/patrickjlong1/sas-llm-paper*
- *Alammar, J., & Grootendorst, M. (2024). Hands-on large language models: Language understanding and generation. O'Reilly Media*

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the authors at:

Patrick Long
US Bureau of Labor Statistics
long.patrick@bls.gov

Brandon Harris
US Bureau of Labor Statistics
harris.brandon@bls.gov